



Що таке Kubernetes?

Що таке Kubernetes?



Kubernetes — це портативна, розширювана платформа з відкритим вихідним кодом для керування контейнерними робочими навантаженнями та службами, яка полегшує як декларативне налаштування, так і автоматизацію. Він має велику екосистему, що швидко розвивається. Сервіси, підтримка та інструменти Kubernetes широко доступні.

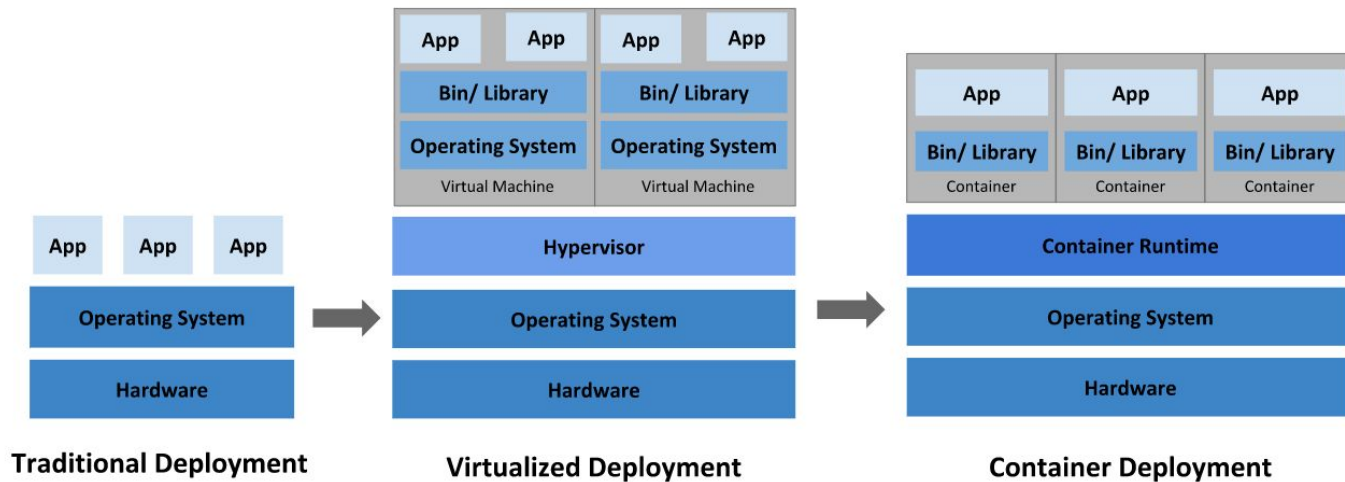
Назва Kubernetes походить від грецького, що означає керманич або пілот. Google відкрила проект Kubernetes у 2014 році. Kubernetes поєднує в собі понад 15 років досвіду Google у створенні робочих навантажень у великих масштабах із найкращими у своєму класі ідеями та практиками спільноти.

Kubernetes в інструменті керування контейнерами з відкритим вихідним кодом, розміщеному Cloud Native Computing Foundation (CNCF). Це також відоме як розширена версія Borg, розроблена в Google для керування як тривалими процесами, так і пакетними завданнями, які раніше оброблялися окремими системами.

Kubernetes має можливість автоматизувати розгортання, масштабування програми та роботу контейнерів програм у кластерах. Він здатний створювати інфраструктуру, орієнтовану на контейнери.

Повертаючись у минуле

Давайте подивимося, чому Kubernetes такий корисний, повернувшись у минуле.



Повертаючись у минуле

Традиційна ера розгортання: на початку організації запускали програми на фізичних серверах. Не було можливості визначити межі ресурсів для програм на фізичному сервері, і це викликало проблеми з розподілом ресурсів. Наприклад, якщо кілька програм працюють на фізичному сервері, можуть бути випадки, коли одна програма займе більшу частину ресурсів, і в результаті інші програми працюватимуть погано. Рішенням для цього було б запустити кожен програму на іншому фізичному сервері. Але це не мало масштабу, оскільки ресурси використовувалися недостатньо, а для організацій було дорого підтримувати багато фізичних серверів.

Епоха віртуалізованого розгортання: як рішення було введено віртуалізацію. Це дозволяє запускати кілька віртуальних машин (ВМ) на одному фізичному ЦП сервера. Віртуалізація дозволяє ізолювати програми між віртуальними машинами та забезпечує певний рівень безпеки, оскільки одна програма не може отримати вільний доступ до інформації іншої програми.

Повертаючись у минуле

Віртуалізація дозволяє краще використовувати ресурси фізичного сервера та забезпечує кращу масштабованість, оскільки додаток можна легко додавати або оновлювати, зменшує витрати на апаратне забезпечення та багато іншого. За допомогою віртуалізації ви можете представити набір фізичних ресурсів як кластер одноразових віртуальних машин.

Кожна віртуальна машина — це повноцінна машина, на якій працюють усі компоненти, включаючи власну операційну систему, поверх віртуалізованого апаратного забезпечення.

Епоха розгортання контейнерів: контейнери схожі на віртуальні машини, але вони мають послаблені властивості ізоляції, щоб спільно використовувати операційну систему (ОС) між програмами. Тому контейнери вважаються легкими. Подібно до віртуальної машини, контейнер має власну файлову систему, частку ЦП, пам'ять, простір для процесу тощо. Оскільки вони відокремлені від базової інфраструктури, вони переносяться між хмарами та дистрибутивами ОС.

Повертаючись у минуле

Контейнери стали популярними, оскільки вони надають додаткові переваги, такі як:

- Гнучке створення та розгортання додатків: підвищення простоти та ефективності створення образу контейнера порівняно з використанням образу віртуальної машини.
- Безперервна розробка, інтеграція та розгортання: забезпечує надійне та часте створення та розгортання образу контейнера зі швидким та ефективним відкотом (через незмінність образу).
- Відокремлення розробників і операцій: створюйте образи контейнерів додатків під час створення/випуску, а не під час розгортання, таким чином відокремлюючи додатки від інфраструктури.
- Спостережливість не лише відображає інформацію та показники на рівні ОС, але й справність програм та інші сигнали.
- Узгодженість навколишнього середовища під час розробки, тестування та виробництва: працює так само на ноутбучі, що й у хмарі.

Повертаючись у минуле

Більше переваг:

- Портативність розповсюдження в хмарі та ОС: працює на Ubuntu, RHEL, CoreOS, локально, у основних загальнодоступних хмарах та де завгодно.
- Керування, орієнтоване на програму: підвищує рівень абстракції від запуску ОС на віртуальному обладнанні до запуску програми в ОС з використанням логічних ресурсів.
- Слабко пов'язані, розподілені, еластичні, звільнені мікросервіси: програми розбиті на менші, незалежні частини, і ними можна динамічно розгортати та керувати ними, а не монолітним стеком, що працює на одній великій одноцільовій машині.
- Ізоляція ресурсів: передбачувана продуктивність програми.
- Використання ресурсів: висока ефективність і щільність.

Навіщо потрібен Kubernetes і що він може

Контейнери — це хороший спосіб об'єднати та запускати ваші програми. У виробничому середовищі вам потрібно керувати контейнерами, які запускають програми, і гарантувати, що немає простоїв.

Наприклад, якщо контейнер виходить з ладу, необхідно запустити інший контейнер. Чи не було б легше, якби цю поведінку обробляла система?

Ось як Kubernetes приходить на допомогу! Kubernetes надає вам структуру для стійкої роботи розподілених систем. Він піклується про масштабування та відновлення після відмови вашої програми, надає шаблони розгортання тощо. Наприклад, Kubernetes може легко керувати розгортанням Canary для вашої системи.

Kubernetes надає вам:

- **Виявлення служб і балансування навантаження** Kubernetes може розкривати контейнер за допомогою імені DNS або власної IP-адреси. Якщо трафік до контейнера великий, Kubernetes може балансувати навантаження та розподіляти мережевий трафік, щоб розгортання було стабільним.
- **Оркестровка сховищ** Kubernetes дозволяє автоматично монтувати систему зберігання на ваш вибір, наприклад локальні сховища, публічні хмарні постачальники тощо.

Навіщо потрібен Kubernetes і що він може

Kubernetes надає вам:

- **Автоматичне розгортання та відкат** Ви можете описати бажаний стан своїх розгорнутих контейнерів за допомогою Kubernetes, і він може змінити фактичний стан на бажаний із контрольованою швидкістю. Наприклад, ви можете автоматизувати Kubernetes для створення нових контейнерів для вашого розгортання, видалення існуючих контейнерів і адаптації всіх їхніх ресурсів до нового контейнера.
- **Автоматичне пакування в контейнер** Ви надаєте Kubernetes кластер вузлів, які він може використовувати для виконання контейнерних завдань. Ви повідомляєте Kubernetes, скільки процесора та пам'яті (RAM) потрібно кожному контейнеру. Kubernetes може розмістити контейнери на ваших вузлах, щоб якнайкраще використовувати ваші ресурси.
- **Kubernetes із самовідновленням** перезапускає контейнери, які вийшли з ладу, замінює контейнери, знищує контейнери, які не реагують на визначену користувачем перевірку працездатності, і не рекламує їх клієнтам, доки вони не будуть готові до обслуговування.
- **Управління секретом і конфігурацією** Kubernetes дозволяє зберігати та керувати конфіденційною інформацією, як-от паролі, маркери OAuth і ключі SSH. Ви можете розгортати та оновлювати секрети та конфігурацію програми без перебудови образів контейнерів і без розкриття секретів у конфігурації стека.

Чим Kubernetes не є

Kubernetes не є традиційною комплексною системою PaaS (платформа як послуга). Оскільки Kubernetes працює на рівні контейнера, а не на апаратному рівні, він надає деякі загальноприйнятні функції, загальні для пропозицій PaaS, як-от розгортання, масштабування, балансування навантаження, і дозволяє користувачам інтегрувати свої рішення для журналювання, моніторингу та оповіщення. Однак Kubernetes не є монолітним, і ці рішення за замовчуванням є необов'язковими та підключаються. Kubernetes надає будівельні блоки для створення платформ розробників, але зберігає вибір користувача та гнучкість там, де це важливо.

Kubernetes:

- Не обмежує типи підтримуваних програм. Kubernetes прагне підтримувати надзвичайно різноманітні робочі навантаження, включаючи робочі навантаження без збереження стану, із збереженням стану та навантаження обробки даних. Якщо програма може працювати в контейнері, вона повинна чудово працювати на Kubernetes.
- Не розгортає вихідний код і не створює вашу програму. Робочі процеси безперервної інтеграції, доставки та розгортання (CI/CD) визначаються організаційною культурою та перевагами, а також технічними вимогами.

Чим Kubernetes не є

Kubernetes:

- Не надає служб прикладного рівня, таких як проміжне програмне забезпечення (наприклад, шини повідомлень), інфраструктури обробки даних (наприклад, Spark), бази даних (наприклад, MySQL), кеші чи кластерні системи зберігання (наприклад, Ceph) як вбудовані служби. Такі компоненти можуть працювати на Kubernetes і/або до них можуть отримати доступ програми, що працюють на Kubernetes, через переносні механізми, такі як Open Service Broker.
- Не вимагає рішень щодо реєстрації, моніторингу чи оповіщення. Він забезпечує деякі інтеграції як доказ концепції та механізми для збору та експорту показників.
- Не надає та не вимагає мови/системи конфігурації (наприклад, Jsonnet). Він надає декларативний API, на який можуть націлюватися довільні форми декларативних специфікацій.
- Не надає та не приймає жодних комплексних систем конфігурації, обслуговування, керування чи самовідновлення машини.
- Крім того, Kubernetes — це не просто оркестровка. Фактично, це усуває потребу в оркестровці. Технічне визначення оркестровки — це виконання певного робочого процесу: спочатку виконайте А, потім В, потім С. На відміну від цього, Kubernetes складається з набору незалежних процесів керування, які постійно переміщують поточний стан у заданий бажаний стан. Не має значення, як ви доберетеся від А до С. Централізований контроль також не потрібен. Це призводить до створення системи, яка є простішою у використанні та більш потужною, надійною, стійкою та розширюваною.

Особливості Kubernetes

Нижче наведено деякі важливі функції Kubernetes.

- Постійний розвиток, інтеграція та розгортання
- Контейнерна інфраструктура
- Управління, орієнтоване на застосування
- Автоматично масштабована інфраструктура
- Узгодженість середовища під час тестування розробки та виробництва
- Слабко пов'язана інфраструктура, де кожен компонент може діяти як окрема одиниця
- Вища щільність використання ресурсів
- Передбачувана інфраструктура, яка буде створена

Одним із ключових компонентів Kubernetes є те, що він може запускати програми на кластерах інфраструктури фізичних і віртуальних машин. Він також має можливість запускати програми в хмарі.

Це допомагає перейти від інфраструктури, орієнтованої на хост, до інфраструктури, орієнтованої на контейнери.



Компоненти Kubernetes

Компоненти Kubernetes

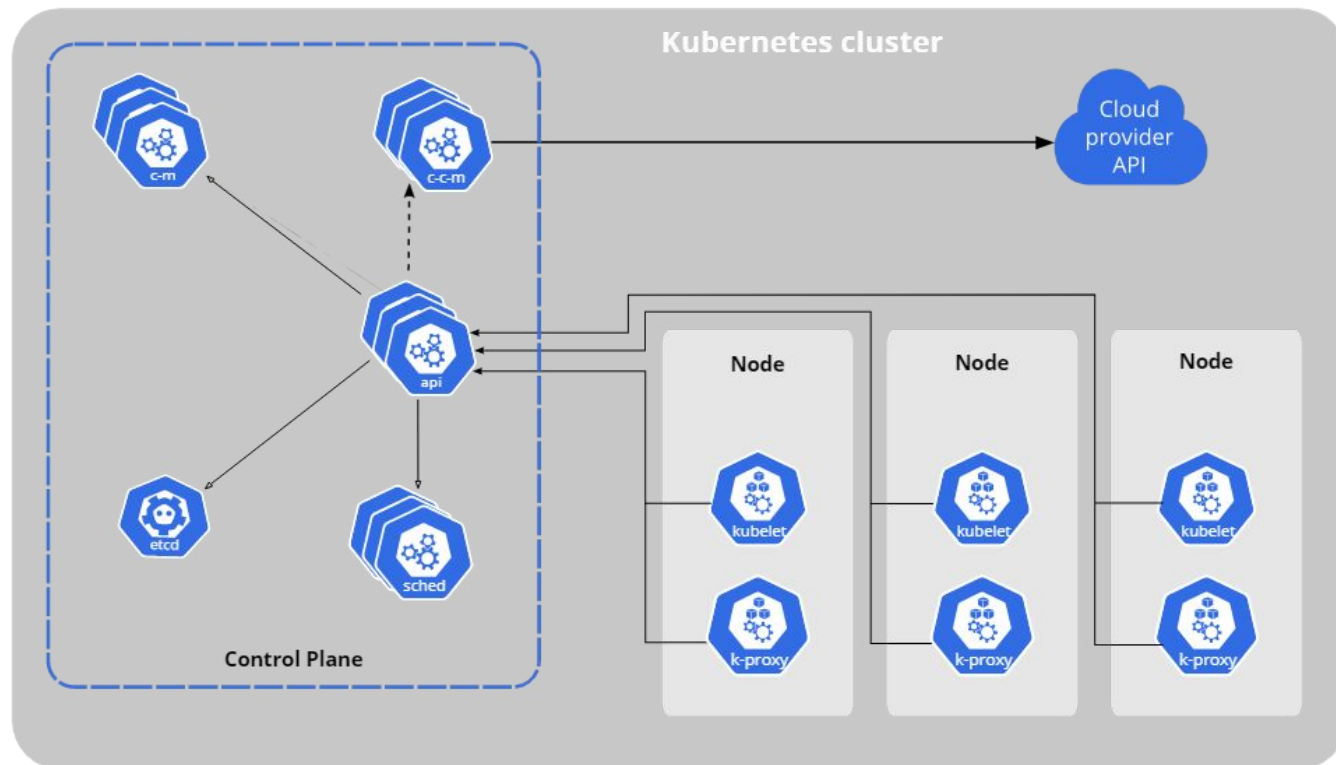
Коли ви розгортаєте Kubernetes, ви отримуєте кластер.

Кластер Kubernetes складається з набору робочих машин, які називаються **вузлами**, які запускають контейнерні програми. Кожен кластер має принаймні один робочий вузол.

Робочі вузли розміщують **модулі**, які є компонентами робочого навантаження програми. Площина керування керує робочими вузлами та модулями в кластері. У виробничих середовищах рівень керування зазвичай працює на кількох комп'ютерах, а кластер зазвичай запускає кілька вузлів, забезпечуючи відмовостійкість і високу доступність.

У цьому документі описано різні компоненти, необхідні для створення повного та робочого кластера Kubernetes.

Ось діаграма кластера Kubernetes з усіма компонентами, пов'язаними разом.



Компоненти площини керування

Компоненти площини керування приймають глобальні рішення щодо кластера (наприклад, планування), а також виявляють події кластера та реагують на них (наприклад, запуск нового модуля, коли поле репліки розгортання не задовольняється).

Компоненти площини керування можна запускати на будь-якій машині в кластері. Однак для спрощення сценарії налаштування зазвичай запускають усі компоненти рівня керування на одній машині та не запускають контейнери користувача на цій машині.

kube-apiserver

Сервер API є компонентом площини керування Kubernetes, який надає доступ до API Kubernetes. Сервер API є передньою частиною для площини керування Kubernetes.

Основною реалізацією сервера Kubernetes API є kube-apiserver. kube-apiserver розроблено для горизонтального масштабування, тобто він масштабується шляхом розгортання більшої кількості екземплярів. Ви можете запустити кілька примірників kube-apiserver і збалансувати трафік між цими примірниками.

Компоненти площини керування

etcd

Послідовне та високодоступне сховище ключових значень, яке використовується як резервне сховище Kubernetes для всіх даних кластера.

Якщо ваш кластер Kubernetes використовує etcd як резервне сховище, переконайтеся, що у вас є план резервного копіювання цих даних.

kube-scheduler

Компонент контрольної площини, який спостерігає за новоствореними контейнерами без призначеного вузла та вибирає вузол для їх запуску.

Фактори, які враховуються для прийняття рішень щодо планування, включають: індивідуальні та колективні вимоги до ресурсів, обмеження апаратного/програмного забезпечення/політики, специфікації спорідненості та антиспорідненості, локальність даних, перешкоди між робочими навантаженнями та кінцеві терміни.

Компоненти площини керування

kube-controller-manager

Компонент Control Plane, який запускає процеси контролера.

Логічно кожен контролер є окремим процесом, але для зменшення складності всі вони скомпільовані в один двійковий файл і виконуються в одному процесі.

Ці контролери включають:

- Контролер вузлів: відповідає за помічання та реагування, коли вузли виходять з ладу.
- Контролер реплікації: відповідає за підтримку правильної кількості контейнерів для кожного об'єкта контролера реплікації в системі.
- Контролер кінцевих точок: заповнює об'єкт Endpoints (тобто об'єднує служби та модулі).
- Контролери облікових записів служби та маркерів: створюйте облікові записи за замовчуванням і маркери доступу до API для нових просторів імен.

Компоненти площини керування

cloud-controller-manager

Компонент площини керування Kubernetes, який вбудовує спеціальну хмарну логіку керування. Менеджер хмарного контролера дозволяє підключити ваш кластер до API вашого хмарного постачальника та відокремлює компоненти, які взаємодіють із цією хмарною платформою, від компонентів, які взаємодіють лише з вашим кластером.

Cloud-controller-manager запускає лише контролери, які є специфічними для вашого хмарного постачальника. Якщо ви використовуєте Kubernetes у себе або в навчальному середовищі на власному ПК, кластер не має диспетчера хмарного контролера.

Як і у випадку з kube-controller-manager, cloud-controller-manager об'єднує кілька логічно незалежних циклів керування в один двійковий файл, який ви запускаєте як єдиний процес. Ви можете масштабувати горизонтально (запустити більше однієї копії), щоб підвищити продуктивність або допомогти витримати збої.

Наступні контролери можуть мати залежності від хмарного постачальника:

- Контролер вузла: для перевірки хмарного постачальника, щоб визначити, чи вузол було видалено в хмарі після того, як він перестав відповідати
- Контролер маршруту: для налаштування маршрутів у базовій хмарній інфраструктурі
- Контролер служби: для створення, оновлення та видалення балансувальників навантаження хмарних постачальників

Компоненти вузла

Компоненти вузла працюють на кожному вузлі, підтримуючи запуснені модулі та забезпечуючи середовище виконання Kubernetes.

kubelet

Агент, який працює на кожному вузлі в кластері. Він гарантує, що контейнери працюють у Pod. Kubelet використовує набір PodSpec, які надаються через різні механізми, і гарантує, що контейнери, описані в цих PodSpec, працюють і справні. Kubelet не керує контейнерами, створеними не Kubernetes.

kube-proxy

kube-proxy — це мережевий проксі, який працює на кожному вузлі вашого кластера, реалізуючи частину концепції служби Kubernetes.

kube-proxy підтримує мережеві правила на вузлах. Ці мережеві правила дозволяють мережевий зв'язок із вашими модулями під час мережевих сеансів у вашому кластері чи поза ним.

kube-proxy використовує рівень фільтрації пакетів операційної системи, якщо вони є та доступні. В іншому випадку kube-proxy пересилає трафік самостійно.

Компоненти вузла

Container runtime

Середовище виконання контейнера — це програмне забезпечення, яке відповідає за запуск контейнерів. Kubernetes підтримує кілька середовищ виконання контейнерів: Docker, containerd, CRI-O та будь-яку реалізацію Kubernetes CRI (Container Runtime Interface).

Додатки

Додатки використовують ресурси Kubernetes (DaemonSet, Deployment тощо) для реалізації функцій кластера. Оскільки вони надають функції на рівні кластера, ресурси простору імен для аддонів належать до простору імен системи kube.

Вибрані додатки описані нижче.

DNS

Хоча інші аддони не обов'язкові, усі кластери Kubernetes повинні мати кластерний DNS, оскільки багато прикладів покладаються на нього. Кластерний DNS – це DNS-сервер на додаток до інших DNS-серверів у вашому середовищі, який обслуговує записи DNS для служб Kubernetes. Контейнери, створені Kubernetes, автоматично включають цей DNS-сервер у свої пошуки DNS.

Додатки

Web UI (Dashboard)

Інформаційна панель — це веб-інтерфейс загального призначення для кластерів Kubernetes. Це дозволяє користувачам керувати програмами, запущеними в кластері, а також самим кластером і усувати неполадки в них.

Моніторинг ресурсів контейнерів

Container Resource Monitoring записує загальні показники часових рядів про контейнери в центральній базі даних і надає інтерфейс користувача для перегляду цих даних.

Ведення журналів на рівні кластера

Механізм журналювання на рівні кластера відповідає за збереження журналів контейнерів у центральному сховищі журналів з інтерфейсом пошуку/перегляду.

Питання та відповіді